WEEK 16

Final Project, Part 2: Finish, Test and Publish

Daily study material with detailed examples

Android Development with Kotlin | Prepared for **Anjana**

Last week you built the core of the Expense Tracker. This week you finish it like a professional: a summary screen with a chart, settings and a budget, tests, a signed release build, the Play Store steps and a README that shows off your work.

Day	Topic	You will be able to...
Day 1	Summary screen and bottom navigation	Show a month's spending by category, with a chart and currency conversion
Day 2	Settings, budget and polish	Save settings with DataStore and make the app feel finished
Day 3	Test the project	Write unit and ViewModel tests for your own app
Day 4	Release build and signing	Create a keystore and build a signed AAB and APK
Day 5	Play Store basics	Know every step from developer account to production
Day 6	README, portfolio and next steps	Present your project on GitHub and plan what comes next

Tip: As in Week 15, the plain Kotlin code and all the **tests were run**, so the outputs are real (the tests used small stand-ins for JUnit and kotlinx-coroutines-test that behave the same way for these examples). The Android code was compile-checked against stand-ins for the libraries that couldn't be downloaded here. Gradle, signing and Play Console steps are Android-only and weren't run, so follow Android Studio's screens if they look slightly different.

WEEK 16 • DAY 1

Summary Screen and Bottom Navigation

Today's goal: Show the month's spending by category with a donut chart and bars, convert the total with the API, and add a bottom bar.

1. What the summary screen shows

Sketch: Summary screen

```
+--------------------------------+
|  <      March 2025          >  |
|  Total spent                   |
|  Rs. 1,299.00                  |
|  4 expenses                    |
|  [Show in US dollars]          |
|                                |
|          (  donut  )           |
|                                |
|  Travel     Rs. 500.00  (38%)  |
|  ##################            |
|  Food       Rs. 400.00  (30%)  |
|  ###############               |
|  Bills      Rs. 399.00  (30%)  |
|  ###############               |
+--------------------------------+
| Expenses | Summary | Settings  |
+--------------------------------+
```

The numbers come from **summarize()**, which you wrote and ran in Week 15. The new parts are moving between months and the currency conversion.

2. The conversion logic

Turning a Result into screen state (run)

```
sealed interface ConversionState {
    data object Idle : ConversionState
    data object Loading : ConversionState
    data class Done(val text: String) : ConversionState
    data class Failed(val message: String) : ConversionState
}

fun conversionState(totalPaise: Long, currency: String, rate: Result<Double>): ConversionState =
    rate.fold(
        onSuccess = { ConversionState.Done("About $currency " + "%.2f".format(totalPaise / 100.0 * it)) },
        onFailure = { ConversionState.Failed("Couldn't get the rate. Check your internet and try again.") },
    )

import java.io.IOException

fun main() {
    println(conversionState(1_250_000, "USD", Result.success(0.01148)))
    println(conversionState(1_250_000, "USD", Result.failure(IOException("No internet"))))
}
```

Output

```
Done(text=About USD 143.50)
Failed(message=Couldn't get the rate. Check your internet and try again.)
```

fold handles both cases of a Result in one expression. The screen will show a button (Idle), a spinner (Loading), the text (Done) or the error with a Retry button (Failed).

3. Months and YearMonth

Moving between months (run)

```
import java.time.YearMonth

fun main() {
    val current = YearMonth.of(2025, 1)
    val previous = current.minusMonths(1)
    println(previous)                          // the year changes automatically
    println(previous < current)                // YearMonth can be compared
    println(current.lengthOfMonth())
    println(YearMonth.of(2024, 2).lengthOfMonth())   // leap year
}
```

Output

```
2024-12
true
31
29
```

4. The ViewModel

SummaryViewModel.kt (ui.summary)

```
data class SummaryUiState(
    val summary: MonthSummary,
    val canGoNext: Boolean,
    val conversion: ConversionState = ConversionState.Idle,
)

@HiltViewModel
class SummaryViewModel @Inject constructor(
    repository: ExpenseRepository,
    private val currencyRepository: CurrencyRepository,
    todayProvider: TodayProvider,
) : ViewModel() {

    private val currentMonth = YearMonth.from(todayProvider.today())
    private val selectedMonth = MutableStateFlow(currentMonth)
    private val conversion = MutableStateFlow<ConversionState>(ConversionState.Idle)

    val uiState: StateFlow<SummaryUiState> =
        combine(repository.observeExpenses(), selectedMonth, conversion) { expenses, month, conv ->
            SummaryUiState(summarize(expenses, month), canGoNext = month < currentMonth, conversion = conv)
        }.stateIn(
            viewModelScope,
            SharingStarted.WhileSubscribed(5_000),
            SummaryUiState(MonthSummary(currentMonth, 0, 0, emptyList()), canGoNext = false),
        )

    fun previousMonth() {
        selectedMonth.update { it.minusMonths(1) }
        conversion.value = ConversionState.Idle        // the old conversion was for another month
    }

    fun nextMonth() {
        if (selectedMonth.value < currentMonth) {
            selectedMonth.update { it.plusMonths(1) }
            conversion.value = ConversionState.Idle
        }
    }
```

```
    fun convertTo(currency: String) {
        val totalPaise = uiState.value.summary.totalPaise
        conversion.value = ConversionState.Loading
        viewModelScope.launch {
            conversion.value = conversionState(totalPaise, currency, currencyRepository.rateFromInr(currency))
        }
    }
}
```

Piece	Why
combine(expenses, month, conversion)	Any of the three changing creates a new UiState
canGoNext = month < currentMonth	You can't go into the future, so the ">" button is disabled on this month
conversion = Idle when the month changes	An old "About USD 15.60" must not stay on screen for another month
uiState.value.summary.totalPaise	Converts exactly the total that's on the screen

5. A chart with Canvas

You don't need a chart library for a simple chart. **Canvas** gives you a DrawScope where you draw arcs, lines and rectangles yourself. Each category's arc is its share of 360 degrees.

Chart.kt (ui.summary)

```
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.geometry.Size
import androidx.compose.ui.graphics.drawscope.Stroke

fun categoryColor(category: Category): Color = when (category) {
    Category.FOOD -> Color(0xFFEF6C00)
    Category.TRAVEL -> Color(0xFF1E88E5)
    Category.SHOPPING -> Color(0xFF8E24AA)
    Category.BILLS -> Color(0xFF43A047)
    Category.HEALTH -> Color(0xFFE53935)
    Category.OTHER -> Color(0xFF757575)
}

@Composable
fun DonutChart(totals: List<CategoryTotal>, modifier: Modifier = Modifier) {
    val sum = totals.sumOf { it.totalPaise }.toFloat()
    Canvas(modifier) {
        val stroke = 36f
        var start = -90f                                    // start at 12 o'clock
        totals.forEach { t ->
            val sweep = 360f * t.totalPaise / sum             // this category's slice of the circle
            drawArc(
                color = categoryColor(t.category),
                startAngle = start,
                sweepAngle = sweep,
                useCenter = false,
                topLeft = Offset(stroke / 2, stroke / 2),
                size = Size(size.width - stroke, size.height - stroke),
                style = Stroke(width = stroke),
            )
            start += sweep
        }
    }
}

@Composable
fun CategoryBar(item: CategoryTotal, maxPaise: Long) {
    Column(Modifier.fillMaxWidth()) {
        Row {
            Text(item.category.label, modifier = Modifier.weight(1f))
            Text("${formatRupees(item.totalPaise)}  (${item.percent}%)")
```

```
        }
        Spacer(Modifier.height(4.dp))
        Box(
            Modifier.fillMaxWidth().height(10.dp).clip(RoundedCornerShape(5.dp))
                .background(MaterialTheme.colorScheme.surfaceVariant)
        ) {
            Box(
                Modifier.fillMaxWidth(item.totalPaise.toFloat() / maxPaise).fillMaxHeight()
                    .background(categoryColor(item.category))
            )
        }
    }
}
```

Tip: Half of a Stroke is drawn outside the shape. That's why the arc is moved in by **stroke / 2** and made smaller by **stroke**; otherwise the edges of the donut get cut off.

6. The screen

SummaryScreen.kt (ui.summary)

```
@Composable
fun SummaryScreen(viewModel: SummaryViewModel = hiltViewModel()) {
    val state by viewModel.uiState.collectAsStateWithLifecycle()
    SummaryContent(
        state = state,
        onPrevious = viewModel::previousMonth,
        onNext = viewModel::nextMonth,
        onConvert = { viewModel.convertTo("USD") },
    )
}

@Composable
fun SummaryContent(state: SummaryUiState, onPrevious: () -> Unit, onNext: () -> Unit, onConvert: () -> Unit) {
    val summary = state.summary
    LazyColumn(
        modifier = Modifier.fillMaxSize(),
        contentPadding = PaddingValues(16.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp),
    ) {
        item { MonthSwitcher(summary.month, state.canGoNext, onPrevious, onNext) }
        item {
            Card(Modifier.fillMaxWidth()) {
                Column(Modifier.padding(16.dp)) {
                    Text("Total spent", style = MaterialTheme.typography.labelLarge)
                    Text(formatRupees(summary.totalPaise), style = MaterialTheme.typography.headlineMedium)
                    Text("${summary.count} expenses")
                    Spacer(Modifier.height(8.dp))
                    ConversionRow(state.conversion, enabled = summary.totalPaise > 0, onConvert = onConvert)
                }
            }
        }
        if (summary.byCategory.isEmpty()) {
            item { Text("No expenses in this month.", modifier = Modifier.padding(16.dp)) }
        } else {
            item {
                Box(Modifier.fillMaxWidth(), contentAlignment = Alignment.Center) {
                    DonutChart(summary.byCategory, Modifier.size(180.dp))
                }
            }
            items(summary.byCategory, key = { it.category }) { item ->
                CategoryBar(item, maxPaise = summary.byCategory.first().totalPaise)
            }
        }
    }
}

@Composable
```

```
fun MonthSwitcher(month: YearMonth, canGoNext: Boolean, onPrevious: () -> Unit, onNext: () -> Unit) {
    Row(Modifier.fillMaxWidth(), verticalAlignment = Alignment.CenterVertically) {
        IconButton(onClick = onPrevious) {
            Icon(Icons.AutoMirrored.Filled.KeyboardArrowLeft, contentDescription = "Previous month")
        }
        Text(
            month.format(java.time.format.DateTimeFormatter.ofPattern("MMMM yyyy")),
            style = MaterialTheme.typography.titleLarge,
            textAlign = TextAlign.Center,
            modifier = Modifier.weight(1f),
        )
        IconButton(onClick = onNext, enabled = canGoNext) {
            Icon(Icons.AutoMirrored.Filled.KeyboardArrowRight, contentDescription = "Next month")
        }
    }
}

@Composable
fun ConversionRow(state: ConversionState, enabled: Boolean, onConvert: () -> Unit) {
    when (state) {
        ConversionState.Idle -> TextButton(onClick = onConvert, enabled = enabled) { Text("Show in US dollars") }
        ConversionState.Loading -> CircularProgressIndicator(Modifier.size(24.dp))
        is ConversionState.Done -> Text(state.text, style = MaterialTheme.typography.bodyLarge)
        is ConversionState.Failed -> Row(verticalAlignment = Alignment.CenterVertically) {
            Text(state.message, color = MaterialTheme.colorScheme.error, modifier = Modifier.weight(1f))
            TextButton(onClick = onConvert) { Text("Retry") }
        }
    }
}
```

Preview

```
@Preview(showBackground = true)
@Composable
fun SummaryPreview() {
    val totals = listOf(
        CategoryTotal(Category.TRAVEL, 50000, 38),
        CategoryTotal(Category.FOOD, 40000, 30),
        CategoryTotal(Category.BILLS, 39900, 30),
    )
    MaterialTheme {
        SummaryContent(
            state = SummaryUiState(MonthSummary(YearMonth.of(2025, 3), 129900, 4, totals), canGoNext = false),
            onPrevious = {}, onNext = {}, onConvert = {},
        )
    }
}
```

7. Bottom navigation

Three top-level screens get a **NavigationBar** (Week 7). The add/edit form is not a tab, so the bar hides there.

ExpenseTrackerApp.kt: the app shell with the bottom bar

```
@Serializable object SummaryRoute
@Serializable object SettingsRoute

data class TopLevelTab(val label: String, val route: Any, val icon: ImageVector)

val topLevelTabs = listOf(
    TopLevelTab("Expenses", ExpenseListRoute, Icons.AutoMirrored.Filled.List),
    TopLevelTab("Summary", SummaryRoute, Icons.Default.DateRange),
    TopLevelTab("Settings", SettingsRoute, Icons.Default.Settings),
)

@Composable
fun ExpenseTrackerApp() {
    val navController = rememberNavController()
```

```
    val backStackEntry by navController.currentBackStackEntryAsState()
    val destination = backStackEntry?.destination
    // Hide the bottom bar on the add/edit form
    val showBottomBar = topLevelTabs.any { tab -> destination?.hasRoute(tab.route::class) == true }

    Scaffold(
        bottomBar = {
            if (showBottomBar) {
                NavigationBar {
                    topLevelTabs.forEach { tab ->
                        NavigationBarItem(
                            selected = destination?.hierarchy?.any { it.hasRoute(tab.route::class) } == true,
                            onClick = {
                                navController.navigate(tab.route) {
                                    popUpTo(navController.graph.findStartDestination().id) { saveState = true }
                                    launchSingleTop = true
                                    restoreState = true
                                }
                            },
                            icon = { Icon(tab.icon, contentDescription = null) },
                            label = { Text(tab.label) },
                        )
                    }
                }
            }
        },
    ) { padding ->
        NavHost(navController, startDestination = ExpenseListRoute, modifier = Modifier.padding(padding)) {
            composable<ExpenseListRoute> {
                ExpenseListScreen(
                    onAddClick = { navController.navigate(AddEditRoute()) },
                    onExpenseClick = { id -> navController.navigate(AddEditRoute(id)) },
                )
            }
            composable<AddEditRoute> { AddEditScreen(onDone = { navController.popBackStack() }) }
            composable<SummaryRoute> { SummaryScreen() }
            composable<SettingsRoute> { SettingsScreen() }
        }
    }
}
```

In MainActivity, replace **ExpenseNavHost()** with **ExpenseTrackerApp()**. SettingsScreen comes tomorrow; for today a placeholder like **@Composable fun SettingsScreen() { Text("Settings") }** is enough.

Navigation option	What it does
popUpTo(start) { saveState = true }	Tabs don't pile up on the back stack; Back from any tab goes to Expenses, then exits
launchSingleTop = true	Tapping the current tab again doesn't open a second copy
restoreState = true	Coming back to a tab restores its scroll position and state

Careful: Each screen has its own Scaffold inside the app's Scaffold. If you see a double gap at the bottom on some phones, pass **contentWindowInsets = WindowInsets(0, 0, 0, 0)** to the inner Scaffolds, because the outer one already handles it.

Practice Exercises

1. Add **fun MonthSummary.averagePaise(): Long** that returns the average amount per expense (0 when there are no expenses), and print it for a small list.

2. Write **fun biggestExpense(expenses: List<Expense>, month: YearMonth): Expense?** that returns the most expensive item of that month, or null. (Hint: **maxByOrNull**.) The summary card could show "Biggest: Phone bill, Rs. 399.00".

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Why does changing the month reset the conversion to Idle?

Q2. What does Result.fold do?

Q3. Why is the stroke arc drawn with topLeft = stroke / 2?

Q4. Why is the bottom bar hidden on the add/edit screen?

WEEK 16 • DAY 2

Settings, Budget and Polish

Today's goal: Save a monthly budget and theme with DataStore, warn when the budget is used up, and polish the app until it feels finished.

1. Budget logic first

Same habit as before: write and run the rule in plain Kotlin, then build the UI around it.

BudgetStatus (run)

```
sealed interface BudgetStatus {
    data object NotSet : BudgetStatus
    data class Under(val leftPaise: Long, val fraction: Float) : BudgetStatus
    data class Over(val overPaise: Long) : BudgetStatus
}

fun budgetStatus(spentPaise: Long, budgetPaise: Long): BudgetStatus = when {
    budgetPaise <= 0 -> BudgetStatus.NotSet
    spentPaise > budgetPaise -> BudgetStatus.Over(spentPaise - budgetPaise)
    else -> BudgetStatus.Under(budgetPaise - spentPaise, spentPaise.toFloat() / budgetPaise)
}

fun main() {
    println(budgetStatus(spentPaise = 300_000, budgetPaise = 0))
    println(budgetStatus(spentPaise = 300_000, budgetPaise = 500_000))
    println(budgetStatus(spentPaise = 620_000, budgetPaise = 500_000))

    val status = budgetStatus(300_000, 500_000)
    if (status is BudgetStatus.Under) println("${formatRupees(status.leftPaise)} left, ${(status.fraction * 100).toInt()}% used")
}
```

Output

```
NotSet
Under(leftPaise=200000, fraction=0.6)
Over(overPaise=120000)
Rs. 2,000.00 left, 60% used
```

2. Settings with DataStore

Small settings like a budget or a theme go in **DataStore** (Week 11), not in Room. A repository hides the keys, so the rest of the app only sees a **Flow<UserSettings>**.

SettingsRepository.kt (data.settings)

```
enum class ThemeMode { SYSTEM, LIGHT, DARK }

data class UserSettings(
    val monthlyBudgetPaise: Long = 0,                  // 0 = no budget set
    val themeMode: ThemeMode = ThemeMode.SYSTEM,
)

class SettingsRepository @Inject constructor(
    private val dataStore: DataStore<Preferences>,
) {
    private object Keys {
        val BUDGET = longPreferencesKey("monthly_budget_paise")
        val THEME = stringPreferencesKey("theme_mode")
    }

    val settings: Flow<UserSettings> = dataStore.data.map { prefs ->
        UserSettings(
            monthlyBudgetPaise = prefs[Keys.BUDGET] ?: 0,
            themeMode = ThemeMode.entries.find { it.name == prefs[Keys.THEME] } ?: ThemeMode.SYSTEM,
        )
    }

    suspend fun setBudget(paise: Long) {
        dataStore.edit { it[Keys.BUDGET] = paise }
    }

    suspend fun setThemeMode(mode: ThemeMode) {
        dataStore.edit { it[Keys.THEME] = mode.name }
    }
}
```

di/DataStoreModule.kt

```
private val Context.settingsDataStore by preferencesDataStore(name = "settings")

@Module
@InstallIn(SingletonComponent::class)
object DataStoreModule {
    @Provides
    @Singleton
    fun provideDataStore(@ApplicationContext context: Context): DataStore<Preferences> = context.settingsDataStore
}
```

Tip: The theme is saved as the enum's **name** and read back with **entries.find**. If the saved text is missing or unknown, it falls back to SYSTEM instead of crashing (unlike valueOf()).

3. The budget on the home screen

ExpenseListViewModel.kt now combines expenses and settings

```
data class ExpenseListUiState(
    val isLoading: Boolean = true,
    val sections: List<DaySection> = emptyList(),
    val monthTotalPaise: Long = 0,
    val budget: BudgetStatus = BudgetStatus.NotSet,            // new
)

@HiltViewModel
class ExpenseListViewModel @Inject constructor(
    private val repository: ExpenseRepository,
    settingsRepository: SettingsRepository,                     // new
    private val todayProvider: TodayProvider,
) : ViewModel() {

    val uiState: StateFlow<ExpenseListUiState> =
        combine(repository.observeExpenses(), settingsRepository.settings) { list, settings ->
            val thisMonth = YearMonth.from(todayProvider.today())
            val monthTotal = list.filter { YearMonth.from(it.date) == thisMonth }.sumOf { it.amountPaise }
            ExpenseListUiState(
                isLoading = false,
                sections = groupByDay(list),
                monthTotalPaise = monthTotal,
                budget = budgetStatus(monthTotal, settings.monthlyBudgetPaise),
            )
        }.stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), ExpenseListUiState())

    // delete() and undoDelete() stay the same as in Week 15
}
```

BudgetCard.kt (ui.list)

```
@Composable
fun BudgetCard(status: BudgetStatus) {
    when (status) {
        BudgetStatus.NotSet -> Unit                     // show nothing
        is BudgetStatus.Under -> Card(Modifier.fillMaxWidth()) {
            Column(Modifier.padding(16.dp)) {
                Text("${formatRupees(status.leftPaise)} left this month")
                Spacer(Modifier.height(8.dp))
                LinearProgressIndicator(progress = { status.fraction }, modifier = Modifier.fillMaxWidth())
            }
        }
        is BudgetStatus.Over -> Card(
            modifier = Modifier.fillMaxWidth(),
            colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.errorContainer),
        ) {
            Text(
                "Over budget by ${formatRupees(status.overPaise)}",
                color = MaterialTheme.colorScheme.onErrorContainer,
                modifier = Modifier.padding(16.dp),
            )
        }
    }
}
```

In ExpenseListContent, add **item { BudgetCard(state.budget) }** right after the MonthTotalCard.

4. The settings screen

SettingsScreen.kt (ui.settings)

```
import androidx.compose.foundation.selection.selectable

@HiltViewModel
```

```
class SettingsViewModel @Inject constructor(
    private val repository: SettingsRepository,
) : ViewModel() {
    val settings: StateFlow<UserSettings> = repository.settings
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), UserSettings())

    // Returns false if the text isn't a valid amount, so the screen can show an error.
    fun saveBudget(text: String): Boolean {
        val paise = if (text.isBlank()) 0L else parseRupees(text) ?: return false
        if (paise < 0) return false
        viewModelScope.launch { repository.setBudget(paise) }
        return true
    }

    fun setThemeMode(mode: ThemeMode) {
        viewModelScope.launch { repository.setThemeMode(mode) }
    }
}

@Composable
fun SettingsScreen(viewModel: SettingsViewModel = hiltViewModel()) {
    val settings by viewModel.settings.collectAsStateWithLifecycle()
    var budgetText by rememberSaveable(settings.monthlyBudgetPaise) {
        mutableStateOf(if (settings.monthlyBudgetPaise > 0) toAmountText(settings.monthlyBudgetPaise) else "")
    }
    var budgetError by remember { mutableStateOf(false) }

    Column(Modifier.fillMaxSize().padding(16.dp), verticalArrangement = Arrangement.spacedBy(12.dp)) {
        Text("Settings", style = MaterialTheme.typography.headlineSmall)

        Text("Monthly budget", style = MaterialTheme.typography.titleMedium)
        OutlinedTextField(
            value = budgetText,
            onValueChange = { budgetText = it; budgetError = false },
            label = { Text("Budget (Rs.), empty = none") },
            isError = budgetError,
            supportingText = { if (budgetError) Text("Enter a number like 5000") },
            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
            singleLine = true,
            modifier = Modifier.fillMaxWidth(),
        )
        Button(onClick = { budgetError = !viewModel.saveBudget(budgetText) }) { Text("Save budget") }

        HorizontalDivider()
        Text("Theme", style = MaterialTheme.typography.titleMedium)
        ThemeMode.entries.forEach { mode ->
            Row(
                Modifier.fillMaxWidth()
                    .selectable(selected = settings.themeMode == mode, onClick = { viewModel.setThemeMode(mode) })
                    .padding(vertical = 8.dp),
                verticalAlignment = Alignment.CenterVertically,
            ) {
                RadioButton(selected = settings.themeMode == mode, onClick = null)
                Spacer(Modifier.width(12.dp))
                Text(mode.name.lowercase().replaceFirstChar { it.uppercase() })     // "System", "Light", "Dark"
            }
        }
    }
}
```

Piece	Why
rememberSaveable(settings.monthlyBudgetPaise)	The text field starts with the saved budget, and resets if the saved value changes
saveBudget() returns Boolean	The screen shows an error when the text isn't a valid amount
Modifier.selectable on the Row	The whole row is tappable, not only the small radio button (better for accessibility)
RadioButton(onClick = null)	The Row handles the click, so the button itself doesn't need to

5. Dark mode from the settings

Theme.kt and MainViewModel.kt

```
@Composable
fun ExpenseTrackerTheme(themeMode: ThemeMode, content: @Composable () -> Unit) {
    val dark = when (themeMode) {
        ThemeMode.SYSTEM -> isSystemInDarkTheme()
        ThemeMode.LIGHT -> false
        ThemeMode.DARK -> true
    }
    MaterialTheme(
        colorScheme = if (dark) darkColorScheme() else lightColorScheme(),
        content = content,
    )
}

@HiltViewModel
class MainViewModel @Inject constructor(settingsRepository: SettingsRepository) : ViewModel() {
    val themeMode: StateFlow<ThemeMode> = settingsRepository.settings
        .map { it.themeMode }
        .stateIn(viewModelScope, SharingStarted.Eagerly, ThemeMode.SYSTEM)
}
```

MainActivity.kt uses the saved theme (Android only)

```
@AndroidEntryPoint
class MainActivity : ComponentActivity() {
    private val mainViewModel: MainViewModel by viewModels()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            val themeMode by mainViewModel.themeMode.collectAsStateWithLifecycle()
            ExpenseTrackerTheme(themeMode) {
                ExpenseTrackerApp()
            }
        }
    }
}
```

Tip: On Android 12 and newer you can use the user's wallpaper colours: **dynamicDarkColorScheme(context)** and **dynamicLightColorScheme(context)**. The theme file Android Studio generated already shows how.

6. Polish: what makes an app feel finished

Area	What to do
App name and icon	res/values/strings.xml app_name; right-click res → New → Image Asset to make a launcher icon
Text in resources	Move screen text into strings.xml and use stringResource(R.string.x). It makes translation (e.g. Malayalam) possible
Accessibility	Every icon button has a contentDescription; touch targets are at least 48dp; test with a large font size
Dark mode	Switch the theme and check every screen is readable
Long and large values	Try a 40-character title and Rs. 9,99,999.99: nothing should overlap
Empty and error states	No expenses, a month with no data, no internet for the conversion
Rotation	Rotate on every screen: no lost input, no reloads

res/values/strings.xml (start of it)

```
<resources>
    <string name="app_name">Expense Tracker</string>
    <string name="expenses">Expenses</string>
    <string name="add_expense">Add expense</string>
    <string name="no_expenses_title">No expenses yet</string>
    <string name="no_expenses_body">Tap + to add your first one.</string>
    <string name="left_this_month">%1$s left this month</string>
</resources>
```

Using string resources in Compose

```
Text(stringResource(R.string.no_expenses_title))
Text(stringResource(R.string.left_this_month, formatRupees(status.leftPaise)))   // fills in %1$s
```

Practice Exercises

1. Add a fourth budget state **Near(leftPaise, fraction)** for when 80% or more of the budget is used (but not over). Test it with 4,500 spent of a 5,000 budget.

2. Add a **conversionCurrency** setting ("USD", "EUR" or "GBP") to UserSettings and SettingsRepository, saved with a stringPreferencesKey and defaulting to "USD".

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Why are the budget and theme in DataStore and not Room?

Q2. Why read the theme with entries.find instead of valueOf?

Q3. How does the home screen get the budget?

Q4. Name two accessibility checks for your app.

WEEK 16 • DAY 3

Test the Project

Today's goal: Write unit tests for the domain rules, the use case and two ViewModels of your own app, and do a manual test pass.

1. What to test in your app

Test file	Tests	Kind
MoneyTest	parseRupees and formatRupees	Pure functions
ValidationTest	Every rule in validate()	Pure functions
SummaryTest	summarize(): totals, months, sorting	Pure functions
SaveExpenseUseCaseTest	Valid forms are saved, invalid ones aren't	Coroutines + fake repository
ExpenseListViewModelTest	Grouping, month total, delete and undo	ViewModel + MainDispatcherRule
SummaryViewModelTest	Month switching and currency conversion	ViewModel + two fakes

All of these go in **app/src/test/**. They run on your computer in seconds, without an emulator. The domain layer is easy to test because you kept Android out of it.

2. Testing pure functions

MoneyTest.kt

```
import org.junit.Assert.assertEquals
import org.junit.Assert.assertNull
import org.junit.Test

class MoneyTest {
    @Test
    fun `whole rupees are converted to paise`() {
        assertEquals(12000L, parseRupees("120"))
    }

    @Test
    fun `one decimal place means tens of paise`() {
        assertEquals(9950L, parseRupees("99.5"))
    }

    @Test
    fun `more than two decimals is rejected`() {
        assertNull(parseRupees("12.345"))
    }

    @Test
    fun `text that is not a number is rejected`() {
        assertNull(parseRupees("abc"))
    }

    @Test
    fun `format adds separators and two decimals`() {
        assertEquals("Rs. 1,250.05", formatRupees(125005))
    }
}
```

Output

```
MoneyTest
  PASSED  format adds separators and two decimals
  PASSED  more than two decimals is rejected
  PASSED  one decimal place means tens of paise
  PASSED  text that is not a number is rejected
  PASSED  whole rupees are converted to paise
5 tests, 0 failed
```

ValidationTest.kt

```
import org.junit.Assert.assertEquals
import org.junit.Assert.assertTrue
import org.junit.Test

class ValidationTest {
    private val today = LocalDate.of(2025, 3, 10)
    private fun form(title: String = "Lunch", amount: String = "120", date: LocalDate = today) =
        ExpenseForm(title, amount, Category.FOOD, date)

    private fun errorsOf(f: ExpenseForm): Map<Field, String> =
        (validate(f, today) as? ValidationResult.Invalid)?.errors ?: emptyMap()

    @Test
    fun `a normal expense is valid`() {
        assertEquals(ValidationResult.Valid, validate(form(), today))
    }

    @Test
    fun `blank title is an error`() {
        assertEquals("Enter a title", errorsOf(form(title = "   "))[Field.TITLE])
    }

    @Test
    fun `zero amount is an error`() {
        assertEquals("Amount must be more than zero", errorsOf(form(amount = "0"))[Field.AMOUNT])
    }

    @Test
    fun `future date is an error but today is fine`() {
        assertTrue(Field.DATE in errorsOf(form(date = today.plusDays(1))))
        assertTrue(Field.DATE !in errorsOf(form(date = today)))
    }
}
```

Output

```
ValidationTest
  PASSED  a normal expense is valid
  PASSED  blank title is an error
  PASSED  future date is an error but today is fine
  PASSED  zero amount is an error
4 tests, 0 failed
```

A small helper like **form(...)** with default values keeps every test short: each test changes only the one thing it's about. That's the "Arrange" step from Week 14 made easy.

3. Testing the use case

SaveExpenseUseCaseTest.kt

```
import kotlinx.coroutines.flow.first
import kotlinx.coroutines.test.runTest
import org.junit.Assert.assertEquals
import org.junit.Assert.assertTrue
import org.junit.Test

class SaveExpenseUseCaseTest {
    private val repository = InMemoryExpenseRepository()
    private val today = LocalDate.of(2025, 3, 10)
    private val save = SaveExpenseUseCase(repository) { today }

    @Test
    fun `valid form is saved with paise and a trimmed title`() = runTest {
        save(0, ExpenseForm("  Lunch ", "120.50", Category.FOOD, today))
        val saved = repository.observeExpenses().first().single()
        assertEquals("Lunch", saved.title)
        assertEquals(12050L, saved.amountPaise)
    }

    @Test
    fun `invalid form is not saved`() = runTest {
        val result = save(0, ExpenseForm("", "120", Category.FOOD, today))
        assertTrue(result is ValidationResult.Invalid)
        assertTrue(repository.observeExpenses().first().isEmpty())
    }

    @Test
    fun `saving with an existing id updates instead of adding`() = runTest {
        save(0, ExpenseForm("Lunch", "120", Category.FOOD, today))
        save(1, ExpenseForm("Lunch", "150", Category.FOOD, today))
        val all = repository.observeExpenses().first()
        assertEquals(1, all.size)
        assertEquals(15000L, all[0].amountPaise)
    }
}
```

Output

```
SaveExpenseUseCaseTest
  PASSED  invalid form is not saved
  PASSED  saving with an existing id updates instead of adding
  PASSED  valid form is saved with paise and a trimmed title
3 tests, 0 failed
```

Tip: The InMemoryExpenseRepository you wrote in Week 15 is now your **fake**. Writing it early paid off twice: you could run the app before Room existed, and now you can test without a database.

4. Testing a ViewModel that uses WhileSubscribed

There's one new trick. With **SharingStarted.WhileSubscribed**, the uiState only updates while someone is **collecting** it. On a phone the screen collects it. In a test, you collect it yourself in **backgroundScope**, which is cancelled automatically when the test ends.

ExpenseListViewModelTest.kt (uses MainDispatcherRule from Week 14)

```
import kotlinx.coroutines.launch
import kotlinx.coroutines.test.UnconfinedTestDispatcher
import kotlinx.coroutines.test.runTest
import org.junit.Assert.assertEquals
import org.junit.Rule
```

```
import org.junit.Test

class ExpenseListViewModelTest {
    @get:Rule
    val mainDispatcherRule = MainDispatcherRule()

    private val repository = InMemoryExpenseRepository()
    private val today = LocalDate.of(2025, 3, 10)
    private val viewModel = ExpenseListViewModel(repository) { today }

    @Test
    fun `expenses are grouped by day with this month's total`() = runTest {
        backgroundScope.launch(UnconfinedTestDispatcher(testScheduler)) { viewModel.uiState.collect {} }

        repository.upsert(Expense(title = "Lunch", amountPaise = 12000, category = Category.FOOD, date = today))
        repository.upsert(Expense(title = "Auto", amountPaise = 28000, category = Category.TRAVEL, date = today))
        repository.upsert(Expense(title = "Old", amountPaise = 5000, category = Category.FOOD, date = LocalDate.of(2025, 2, 27)))

        val state = viewModel.uiState.value
        assertEquals(false, state.isLoading)
        assertEquals(listOf(today, LocalDate.of(2025, 2, 27)), state.sections.map { it.date })
        assertEquals(40000L, state.monthTotalPaise)          // February isn't counted
    }

    @Test
    fun `undo brings a deleted expense back`() = runTest {
        backgroundScope.launch(UnconfinedTestDispatcher(testScheduler)) { viewModel.uiState.collect {} }
        repository.upsert(Expense(title = "Lunch", amountPaise = 12000, category = Category.FOOD, date = today))
        val lunch = viewModel.uiState.value.sections[0].expenses[0]

        viewModel.delete(lunch)
        assertEquals(0, viewModel.uiState.value.sections.size)

        viewModel.undoDelete()
        assertEquals(listOf("Lunch"), viewModel.uiState.value.sections.flatMap { it.expenses }.map { it.title })
    }
}
```

Output

```
ExpenseListViewModelTest
  PASSED  expenses are grouped by day with this month's total
  PASSED  undo brings a deleted expense back
2 tests, 0 failed
```

Line	Meaning
backgroundScope.launch(...)	Starts a collector that lives until the test ends, like a screen would
UnconfinedTestDispatcher(testScheduler)	Runs the collector immediately, so values are ready on the next line
ExpenseListViewModel(repository) { today }	No Hilt in unit tests: you pass the fakes into the constructor yourself

5. Testing the conversion with a fake API

SummaryViewModelTest.kt

```
import java.io.IOException
import kotlinx.coroutines.launch
import kotlinx.coroutines.test.UnconfinedTestDispatcher
import kotlinx.coroutines.test.runTest
import org.junit.Assert.assertEquals
import org.junit.Rule
import org.junit.Test

class SummaryViewModelTest {
    @get:Rule
    val mainDispatcherRule = MainDispatcherRule()

    private val repository = InMemoryExpenseRepository()
    private val currency = FakeCurrencyRepository()
    private val viewModel = SummaryViewModel(repository, currency) { LocalDate.of(2025, 3, 10) }

    @Test
    fun `conversion uses the month total`() = runTest {
        backgroundScope.launch(UnconfinedTestDispatcher(testScheduler)) { viewModel.uiState.collect {} }
```

```
        repository.upsert(Expense(title = "Rent", amountPaise = 1_000_000, category = Category.BILLS, date = LocalDate.of(2025, 3, 1)))
        currency.result = Result.success(0.012)

        viewModel.convertTo("USD")

        assertEquals(ConversionState.Done("About USD 120.00"), viewModel.uiState.value.conversion)
        assertEquals(listOf("USD"), currency.requested)
    }

    @Test
    fun `network error shows a failure message`() = runTest {
        backgroundScope.launch(UnconfinedTestDispatcher(testScheduler)) { viewModel.uiState.collect {} }
        currency.result = Result.failure(IOException("offline"))

        viewModel.convertTo("USD")

        assertEquals(true, viewModel.uiState.value.conversion is ConversionState.Failed)
    }

    @Test
    fun `cannot go past the current month`() = runTest {
        backgroundScope.launch(UnconfinedTestDispatcher(testScheduler)) { viewModel.uiState.collect {} }
        viewModel.nextMonth()
        assertEquals("2025-03", viewModel.uiState.value.summary.month.toString())
        viewModel.previousMonth()
        assertEquals("2025-02", viewModel.uiState.value.summary.month.toString())
        assertEquals(true, viewModel.uiState.value.canGoNext)
    }
}
```

Output

```
SummaryViewModelTest
  PASSED  cannot go past the current month
  PASSED  conversion uses the month total
  PASSED  network error shows a failure message
3 tests, 0 failed
```

6. A manual test pass before release

Check	Done?
Fresh install: empty state shows, adding the first expense works	
Add, edit, delete, undo; restart the app; everything is still correct	
Summary: switch months, convert with internet, then try with airplane mode (error + Retry)	
Budget: none, under, over; the card changes colour	
Rotation and dark mode on every screen	
Font size set to the largest in phone settings: nothing is cut off	
Run all unit tests: right-click the test folder → Run Tests (all green)	

Practice Exercises

1. Add a test to MoneyTest: **parseRupees("1,250.50")** should return 125050 (the function removes commas).
2. Write **BudgetTest** with three tests for budgetStatus(): no budget, under budget (check leftPaise) and over budget (check overPaise).

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Why must the test collect uiState when the ViewModel uses WhileSubscribed?

Q2. Why use backgroundScope for that collector?

Q3. How do the tests get a repository without Hilt?

Q4. What does FakeCurrencyRepository let you test that the real API can't?

WEEK 16 • DAY 4

Release Build and Signing

Today's goal: Set the version, shrink the app with R8, create a keystore safely, and build a signed AAB and APK.

1. Debug vs release

	Debug build	Release build
Who uses it	You, while developing	Real users
Signed with	An automatic debug key	Your own upload key (keystore)
Code shrinking (R8)	Off	On: smaller and faster
Debuggable	Yes	No
Build it with	The green Run button	Build → Generate Signed App Bundle or APK

2. Set the version

app/build.gradle.kts (Android only)

```
android {
    namespace = "com.anjana.expensetracker"
    compileSdk = 35                                  // as in your project

    defaultConfig {
        applicationId = "com.anjana.expensetracker"   // unique on the Play Store; never change it after publishing
        minSdk = 26
        targetSdk = 35                               // Play requires a recent targetSdk (it goes up every year)
        versionCode = 1                              // a whole number; +1 for every upload
        versionName = "1.0.0"                        // what users see
    }

    buildTypes {
        release {
            isMinifyEnabled = true                   // R8: remove unused code, shorten names
            isShrinkResources = true                 // remove unused images and layouts
            proguardFiles(getDefaultProguardFile("proguard-android-optimize.txt"), "proguard-rules.pro")
        }
    }
}
```

Field	Rule
applicationId	The app's unique id on the Play Store and on the phone. Pick it once
versionCode	Must be higher for every upload: 1, 2, 3... Users never see it
versionName	For people: 1.0.0, then 1.0.1 for a bug fix, 1.1.0 for a new feature
targetSdk	Check the Play Console's current requirement before you upload (it rises every August)

Careful: R8 renames and removes code. Libraries that use reflection need "keep rules". Retrofit, Room, Hilt and kotlinx.serialization ship their own rules, so usually you add nothing. But **always test the release build on a phone**: a crash that only happens in release is almost always a missing keep rule. Search the error message plus "proguard rules".

3. Create your keystore (upload key)

A **keystore** is a file with your private key. Every release is signed with it, which proves the update really comes from you. In Android Studio: **Build** → **Generate Signed App Bundle or APK** → **Android App Bundle** → **Next** → **Create new...**

Field	What to enter
Key store path	Outside the project folder, e.g. Documents/keys/expense-tracker.jks
Password	A strong password. Write it in a password manager
Alias	upload
Key password	Can be the same as the store password
Validity (years)	25 or more
Certificate	Your name; the other fields are optional

Careful: Never commit the keystore or its passwords to Git, and keep a backup (e.g. in a private cloud drive). With Play App Signing (the default for new apps), Google holds the real app signing key and you only hold the upload key, so if you lose it you can ask Google to reset it. Apps shared outside the Play Store can never be updated without the original key.

4. Signing from Gradle (optional, but what teams do)

keystore.properties (in the project root, NOT committed)

```
storeFile=/home/anjana/Documents/keys/expense-tracker.jks
storePassword=your-store-password
keyAlias=upload
keyPassword=your-key-password
```

app/build.gradle.kts reads it (Android only)

```
import java.util.Properties

val keystoreProps = Properties().apply {
    val file = rootProject.file("keystore.properties")
    if (file.exists()) file.inputStream().use { load(it) }
}

android {
    signingConfigs {
        create("release") {
            storeFile = keystoreProps.getProperty("storeFile")?.let { file(it) }
            storePassword = keystoreProps.getProperty("storePassword")
            keyAlias = keystoreProps.getProperty("keyAlias")
            keyPassword = keystoreProps.getProperty("keyPassword")
        }
    }
    buildTypes {
        release {
            signingConfig = signingConfigs.getByName("release")
            // ...isMinifyEnabled etc. as above
        }
    }
}
```

Add to .gitignore

```
keystore.properties
*.jks
*.keystore
```

5. Build the AAB and the APK

File	Used for	How to build	Where it appears
AAB (.aab)	Uploading to the Play Store	./gradlew bundleRelease	app/build/outputs/bundle/release/
APK (.apk)	Installing directly, GitHub Releases, sharing with testers	./gradlew assembleRelease	app/build/outputs/apk/release/

The Play Store only accepts **AAB** files. From an AAB, Google makes a small APK for each phone type. For friends and for your GitHub page, build an **APK**.

Install the release APK on a phone with USB debugging on

```
adb install -r app/build/outputs/apk/release/app-release.apk
```

- Uninstall the debug version first if Android says the signatures don't match.
- Repeat the manual test pass from Day 3 on the **release** build.
- Check the app size in Settings → Apps. With R8 it should be a few MB.

Practice Exercises

1. Set versionCode = 1 and versionName = "1.0.0", turn on isMinifyEnabled and isShrinkResources, and build a release APK.

2. Create your upload keystore outside the project, back it up, and save the passwords in a password manager.

3. Install the release APK on a real phone and do the full manual test pass. Write down anything that's different from debug.

4. Commit the Gradle changes (but not the keystore or keystore.properties) and check on GitHub that no secret was pushed.

These are hands-on tasks for your own project, so there is no answer key. Tick each one off when it's done.

Quick Quiz

Q1. What's the difference between versionCode and versionName?

Q2. Which file does the Play Store need: AAB or APK?

Q3. What usually causes a crash that only happens in the release build?

Q4. Why must the keystore never be in Git?

WEEK 16 • DAY 5

Play Store Basics

Today's goal: Understand every step from a developer account to a production release, and what to prepare for each one.

1. The journey

Sketch: From code to the Play Store

```
Developer account --> Create app --> App content forms --> Store listing
        |                                                        |
        v                                                        v
  (identity check)                Internal test --> Closed test --> Production
                                   (you + team)    (real testers)   (everyone)
```

2. The developer account

- Go to **play.google.com/console** and sign up with a Google account you'll keep for years.
- There's a **one-time registration fee of US$25** and an **identity verification** (ID document). It can take a few days.
- Choose a **personal** account for your own apps. Companies use an **organisation** account (that needs a D-U-N-S number).
- In a team job, you'll normally be **invited** to the company's account instead of creating one.

Careful: New **personal** accounts must run a **closed test** with real testers before they can publish to production. At the time of writing that's at least **12 testers** who stay opted in for **14 days** in a row. Google changes these rules, so check the Play Console Help Center for the current numbers. Ask classmates and family early!

3. Create the app and fill in App content

Form	For the Expense Tracker
Privacy policy	Required for every app. A simple page saying data stays on the phone, and only a currency code is sent to the exchange rate API. You can host it free with GitHub Pages
App access	All features are available without logging in
Ads	No ads
Content rating	Answer the questionnaire; a finance tool usually gets the lowest age rating
Target audience	18+ (or 13+). Choosing children brings extra rules
Data safety	Says what data leaves the phone and why. Here: no personal data is collected or shared
Other declarations	Answer honestly (for example, the app has no lending or payments)

Tip: "Collected" in the Data safety form means **sent off the phone**. Expenses saved in Room on the device are not "collected". If you later add Firebase or analytics, you must update the form.

4. The store listing

Item	Size / limit	Tip
App name	Up to 30 characters	"Expense Tracker - Daily Budget"
Short description	Up to 80 characters	One sentence about the main benefit
Full description	Up to 4,000 characters	Features as a list; keep it honest and simple
App icon	512 x 512 px PNG	The same design as your launcher icon
Feature graphic	1024 x 500 px	Icon + app name on a coloured background
Phone screenshots	At least 2 (up to 8)	Show the list, the form, the summary and dark mode

A sample short and full description

```
Short: Track daily spending, see where your money goes, and stay within your budget.

Full:
Expense Tracker helps you record what you spend in seconds.
- Add expenses with a category, date and note
- See this month's total and a budget warning
- Monthly summary with a chart by category
- Convert totals to US dollars
- Dark mode
- Your data stays on your phone. No account, no ads.
```

5. Testing tracks and release

Track	Who can install	Use it for
Internal testing	Up to 100 people you add by email	Quick checks with your team; available within minutes
Closed testing	Testers you invite (email list or Google Group)	Real feedback; required for new personal accounts
Open testing	Anyone who joins from the store page	Larger beta before launch
Production	Everyone on the Play Store	The real launch. You can roll out to a percentage first

- Upload the **AAB** to a track, write short release notes ("First release"), then **Send for review**.
- Review can take from a few hours to several days. If it's rejected, the email says why. Fix it and upload again with a higher **versionCode**.
- For each update: change the code, raise versionCode and versionName, build a new AAB, upload, and write what changed.

6. Not ready for the Play Store?

That's completely fine for a learning project. Publish the **APK on GitHub Releases** (tomorrow) so anyone can install it, and add the Play Store later. Recruiters mostly look at your **code and README**.

Practice Exercises

1. Write your app's short description (max 80 characters) and full description.

2. Take at least 4 screenshots on the emulator (camera icon in the Running Devices window): list, form, summary, dark mode.

3. Write a simple privacy policy and publish it with GitHub Pages (Settings → Pages in your repository).

4. If you want to publish: make a list of 12+ people who could be closed testers, and ask them now.

These are hands-on tasks for your own project, so there is no answer key. Tick each one off when it's done.

Quick Quiz

Q1. What's the one-time cost of a Google Play developer account?

Q2. Which testing track is fastest for checking a build with your team?

Q3. What does "collected" mean in the Data safety form?

Q4. You uploaded versionCode 1. What must change for the next upload?

WEEK 16 • DAY 6

README, Portfolio and Next Steps

Today's goal: Present the project on GitHub with a great README, screenshots and a release, and plan what to learn next.

1. Why the README matters

A recruiter or team lead spends about **one minute** on your repository. The README is the first thing they see. In that minute it should show what the app does (screenshots), what you used (tech stack) and how the code is organised.

2. A README template

README.md (in the project root)

```
# Expense Tracker

An Android app to record daily expenses, see a monthly summary by category,
and stay within a budget. Built with Kotlin and Jetpack Compose.

<p>
  <img src="screenshots/list.png" width="200"/>
  <img src="screenshots/add.png" width="200"/>
  <img src="screenshots/summary.png" width="200"/>
  <img src="screenshots/dark.png" width="200"/>
</p>

## Features
- Add, edit and delete expenses (with undo)
- Expenses grouped by day, with this month's total
- Monthly summary with a donut chart and category bars
- Monthly budget with a warning
- Convert totals to US dollars (Frankfurter API)
- Light and dark theme

## Tech stack
- Kotlin, Coroutines, Flow
- Jetpack Compose, Material 3, Navigation (type-safe routes)
- MVVM with a domain layer (use cases), unidirectional data flow
- Room, DataStore, Retrofit + kotlinx.serialization
- Hilt for dependency injection
- JUnit tests for the domain layer and ViewModels

## Architecture
ui (Compose screens + ViewModels) -> domain (models, rules, use cases) -> data (Room, Retrofit, DataStore)

## Download
Get the APK from the [Releases](../../releases) page.

## Build it yourself
Open the project in Android Studio (latest stable) and press Run.

## What I learned
- Storing money safely as paise (Long) instead of Double
- Testing ViewModels with fakes and coroutines
- ...
```

- Make a **screenshots** folder in the project root and put your images there, then commit them.
- GitHub shows README.md on the repository's front page automatically.

- Write the **What I learned** part in your own words. It shows you understand the code, not only that it works.

3. Make a GitHub Release

Tag the version

```
git tag v1.0.0
git push --tags
```

On GitHub: **Releases** → **Draft a new release** → choose tag v1.0.0, title "Expense Tracker 1.0.0", write the features, and **attach app-release.apk**. Now anyone can install your app from the link.

4. Polish your GitHub profile

Where	What to do
Repository "About"	A one-line description and topics: android, kotlin, jetpack-compose, room, hilt
Your profile	Pin this repository (and your best weekly projects) at the top
Profile README	A short intro: "Android developer (Kotlin, Compose)", what you're learning, a link to this app
LICENSE	Add one (Add file → Create new file → LICENSE → choose MIT) so others know they can learn from it
Commit history	Small, clear commits show how you work. That's why you committed after every feature

5. Talking about your project

Practise answering these out loud. In interviews (and in your team's meetings), being able to **explain** your choices matters as much as the code.

Question	A good answer mentions...
Walk me through the architecture.	ui → domain → data layers, ViewModels with StateFlow UiState, repositories behind interfaces, Hilt
Why store money as Long?	Double can't represent 0.1 exactly; paise in a Long are exact
How does the list update after you add an expense?	Room's Flow emits a new list; the ViewModel maps it to UiState; Compose recomposes
How did you test it?	Unit tests for rules and use cases; ViewModel tests with fakes, MainDispatcherRule and backgroundScope
What was the hardest bug?	A real story from your project: what happened, how you found it (Logcat, debugger), how you fixed it
What would you add next?	CSV export, reminders with WorkManager, cloud backup with Firebase, more tests

6. Final checklist

Item	Done?
All Must features work; tested on a real phone (release build)	
Unit tests pass	
No secrets in Git (keystore, passwords, API keys)	
README with screenshots, features, tech stack and architecture	
GitHub Release v1.0.0 with the APK attached	
Repository description, topics and pinned on your profile	
(Optional) Closed test running on the Play Store	

7. What next?

Next step	Why
Build 2-3 more apps of your own	Each one gets faster. Try the Recipe or News app idea, or solve a problem from your own life
Firebase (Auth, Firestore)	Login and cloud data are asked about in many jobs
Deeper Compose	Custom layouts, animations, performance, adaptive layouts for tablets and foldables
Read real code	Google's Now in Android sample app; your team's codebase. Ask seniors to explain one part each week
Kotlin Multiplatform	Share code between Android and iOS
Apply for internships	Your GitHub projects are your resume. Link them in every application

Tip: In your team: offer to take small bugs or UI tasks, read every code review comment carefully, and ask "why" when you don't understand a pattern. That's how the 16 weeks turn into real experience.

Practice Exercises

1. Write the README for your project using the template, with at least 4 screenshots.

2. Create the GitHub Release v1.0.0 and attach the release APK. Install it on a friend's phone from the link.

3. Add a description, topics and a LICENSE, and pin the repository on your profile.

4. Record yourself (or practise with a friend) answering the six interview questions above. Keep each answer under 2 minutes.

These are hands-on tasks for your own project, so there is no answer key. Tick each one off when it's done.

Quick Quiz

Q1. What three things should someone learn from your README in one minute?

Q2. How can people install your app without the Play Store?

Q3. Why add a LICENSE file?

Q4. Why are small, clear commits useful for your portfolio?

Answer Key

Try every exercise yourself **before** you look here. Your solution may be different and still correct, as long as the output is right!

Day 1: Summary Screen and Bottom Navigation

Exercise 1

```
fun MonthSummary.averagePaise(): Long = if (count == 0) 0 else totalPaise / count

fun main() {
    val expenses = listOf(
        Expense(1, "Lunch", 12000, Category.FOOD, LocalDate.of(2025, 3, 3)),
        Expense(2, "Bus pass", 50000, Category.TRAVEL, LocalDate.of(2025, 3, 1)),
        Expense(3, "Dinner", 28000, Category.FOOD, LocalDate.of(2025, 3, 8)),
    )
    println(formatRupees(summarize(expenses, YearMonth.of(2025, 3)).averagePaise()))
    println(formatRupees(summarize(expenses, YearMonth.of(2025, 4)).averagePaise()))
}
```

Output

```
Rs. 300.00
Rs. 0.00
```

Exercise 2

```
import java.time.YearMonth

fun biggestExpense(expenses: List<Expense>, month: YearMonth): Expense? =
    expenses.filter { YearMonth.from(it.date) == month }.maxByOrNull { it.amountPaise }

fun main() {
    val expenses = listOf(
        Expense(1, "Lunch", 12000, Category.FOOD, LocalDate.of(2025, 3, 3)),
        Expense(2, "Phone bill", 39900, Category.BILLS, LocalDate.of(2025, 3, 5)),
        Expense(3, "Laptop", 4_500_000, Category.SHOPPING, LocalDate.of(2025, 2, 10)),
    )
    val big = biggestExpense(expenses, YearMonth.of(2025, 3))
    println(if (big == null) "No expenses" else "Biggest: ${big.title}, ${formatRupees(big.amountPaise)}")
    println(biggestExpense(expenses, YearMonth.of(2025, 4)))
}
```

Output

```
Biggest: Phone bill, Rs. 399.00
null
```

Quiz answers

Q1. The converted text belonged to the old month's total; showing it for another month would be wrong.

Q2. It runs onSuccess with the value or onFailure with the error, and returns what that lambda returns.

Q3. Half the stroke width is drawn outside the shape; moving it in keeps the whole ring inside the Canvas.

Q4. It isn't a top-level destination. Forms usually take the full screen and have their own Back button.

Day 2: Settings, Budget and Polish

Exercise 1

```
sealed interface BudgetStatus {
    data object NotSet : BudgetStatus
    data class Under(val leftPaise: Long, val fraction: Float) : BudgetStatus
    data class Near(val leftPaise: Long, val fraction: Float) : BudgetStatus
    data class Over(val overPaise: Long) : BudgetStatus
}

fun budgetStatus(spentPaise: Long, budgetPaise: Long): BudgetStatus {
    if (budgetPaise <= 0) return BudgetStatus.NotSet
    if (spentPaise > budgetPaise) return BudgetStatus.Over(spentPaise - budgetPaise)
    val fraction = spentPaise.toFloat() / budgetPaise
    val left = budgetPaise - spentPaise
    return if (fraction >= 0.8f) BudgetStatus.Near(left, fraction) else BudgetStatus.Under(left, fraction)
}

fun main() {
    println(budgetStatus(450_000, 500_000))
    println(budgetStatus(100_000, 500_000))
}
```

Output

```
Near(leftPaise=50000, fraction=0.9)
Under(leftPaise=400000, fraction=0.2)
```

Exercise 2

```
data class UserSettings(
    val monthlyBudgetPaise: Long = 0,
    val themeMode: ThemeMode = ThemeMode.SYSTEM,
    val conversionCurrency: String = "USD",
)

class SettingsRepository @Inject constructor(private val dataStore: DataStore<Preferences>) {
    private object Keys {
        val BUDGET = longPreferencesKey("monthly_budget_paise")
        val THEME = stringPreferencesKey("theme_mode")
        val CURRENCY = stringPreferencesKey("conversion_currency")
    }

    val settings: Flow<UserSettings> = dataStore.data.map { prefs ->
        UserSettings(
            monthlyBudgetPaise = prefs[Keys.BUDGET] ?: 0,
            themeMode = ThemeMode.entries.find { it.name == prefs[Keys.THEME] } ?: ThemeMode.SYSTEM,
            conversionCurrency = prefs[Keys.CURRENCY] ?: "USD",
        )
    }

    suspend fun setConversionCurrency(code: String) {
        dataStore.edit { it[Keys.CURRENCY] = code }
    }
    // setBudget() and setThemeMode() as before
}
```

Quiz answers

Q1. They're a few small key-value settings, not a table of records. DataStore is made for that.

Q2. valueOf crashes on missing or unknown text; find returns null, so we can fall back to SYSTEM.

Q3. Its ViewModel combines the expenses Flow with the settings Flow and calculates a BudgetStatus.

Q4. Content descriptions on icon buttons, 48dp touch targets, readable at large font sizes, good contrast in dark mode.

Day 3: Test the Project

Exercise 1

```
import org.junit.Assert.assertEquals
import org.junit.Test

class MoneyCommaTest {
    @Test
    fun `commas are ignored`() {
        assertEquals(125050L, parseRupees("1,250.50"))
    }
}
```

Output

```
MoneyCommaTest
  PASSED  commas are ignored
1 tests, 0 failed
```

Exercise 2

```
import org.junit.Assert.assertEquals
import org.junit.Test

class BudgetTest {
    @Test
    fun `no budget means NotSet`() {
        assertEquals(BudgetStatus.NotSet, budgetStatus(spentPaise = 100_000, budgetPaise = 0))
    }

    @Test
    fun `under budget shows what is left`() {
        val status = budgetStatus(spentPaise = 300_000, budgetPaise = 500_000) as BudgetStatus.Under
        assertEquals(200_000L, status.leftPaise)
    }

    @Test
    fun `over budget shows how much over`() {
        assertEquals(BudgetStatus.Over(120_000), budgetStatus(spentPaise = 620_000, budgetPaise = 500_000))
    }
}
```

Output

```
BudgetTest
  PASSED  no budget means NotSet
  PASSED  over budget shows how much over
  PASSED  under budget shows what is left
3 tests, 0 failed
```

Quiz answers

Q1. Without a collector the shared flow never starts, so uiState.value stays at its initial value.

Q2. It's cancelled automatically at the end of the test, so the never-ending collect doesn't keep the test running.

Q3. They create the fake (InMemoryExpenseRepository) and pass it to the constructor directly.

Q4. Errors on demand (no internet), fixed rates for exact assertions, and which currency was requested.

Day 4: Release Build and Signing

Quiz answers

Q1. versionCode is a whole number that must increase with every upload; versionName is the text users see, like 1.0.0.

Q2. AAB (Android App Bundle).

Q3. R8 removed or renamed code that's used by reflection, so a keep rule is missing.

Q4. Anyone with it (and the passwords) could sign updates as you. It's a secret, like a password.

Day 5: Play Store Basics

Quiz answers

Q1. US$25 (plus identity verification).

Q2. Internal testing.

Q3. Data that is sent off the device. Data saved only on the phone isn't collected.

Q4. versionCode must be higher (2 or more); versionName usually changes too.

Day 6: README, Portfolio and Next Steps

Quiz answers

Q1. What the app does (screenshots), the tech stack, and how the code is organised.

Q2. Download the APK from your GitHub Release.

Q3. It tells others what they're allowed to do with your code; without one, the default is "all rights reserved".

Q4. They show how you work step by step, and make the history easy to read and review.

Next: You've finished the whole 16-week course! Send your mentor the link to your GitHub repository, and keep building: your next app will be faster and better. Congratulations, Anjana!